

SIMATS ENGINEERING
Department of Computer Science & Engineering
ASSESSMENT TOOL 1-ANALYTICAL PROBLEM SOLVING

Course Code:	CSA12	Course Title:	Computer Architecture
CO Assessed:	CO2	Assessment:	ASSESSMENT TOOL1- ANALYTICAL PROBLEM SOLVING
Weightage:	45%	Total Marks:	25
CO2Statement:	Apply integer and floating-point arithmetic algorithms including Booth's multiplication, restoring/non-restoring division, and IEEE 754 standard operations to solve fixed-point and floating-point computational problems. (BL3)		
Student Name:	Jeevan Raaja S	Reg.No.:	192421005
Department:	Information Technology	Date:	09/08/2026

ANNEXURE A

1. A processor performs arithmetic operations on signed 8-bit integers using 2's complement representation. Consider two numbers: +45 and -23. Analyze in detail how the system performs both addition and subtraction, including binary conversion, alignment of sign bits, and intermediate steps. Determine whether overflow occurs, and explain how the processor detects overflow and interprets the final result.
2. A high-performance computing system replaces a ripple carry adder with a 4-bit carry look-ahead adder (CLA) to reduce computation delay. Given two binary inputs, analyze how generate and propagate signals are formed and how carries are computed in advance. Compare the time delay of CLA with ripple carry addition and justify, with reasoning, why CLA improves speed in modern processors.
3. In a signed multiplication operation, a processor uses Booth's algorithm to multiply two numbers, -6 and +5. Analyze the algorithm step-by-step, including the role of the accumulator, multiplier, and Booth bit, along with arithmetic shifts. Explain how the algorithm efficiently handles signed numbers and reduces the number of addition/subtraction operations compared to traditional multiplication.
4. A processor is required to perform division of two binary numbers (e.g., $13 \div 3$) using both restoring and non-restoring division algorithms. Analyze both methods step-by-step, showing intermediate remainders and quotient formation. Compare the two approaches in terms of number of steps, complexity, and efficiency, and explain why non-restoring division is often preferred in modern systems.
5. A scientific application requires precise handling of real numbers using the IEEE 754 standard for floating-point representation. Given a decimal number (e.g., -13.25), analyze how it is represented in both single precision and double precision formats, including sign, exponent, and mantissa fields. Further, explain how floating-point addition is performed between two such numbers, highlighting issues like normalization, rounding, and precision loss.

Code of Conduct:

I JEEVAN RAAJA S (192421005) certify that this submission is my original work and that I have adhered to the guidelines specified for this assessment. I understand that any violation of academic integrity rules will result in disciplinary action.

Signature of the student:
JEEVAN RAAJA S

MARKS ALLOCATION

	Problem Understanding(20%)	Method Selection(20%)	Solution Process(25%)	Accuracy of Calculation(20%)	Interpretation of Results(15%)
Q1					
Q2					
Q3					
Q4					
Q5					

RUBRICS FOR CALCULATION:

Criteria	Weightage	Level 4 – Exemplary	Level 3 – Proficient	Level 2 – Developing	Level 1 – Beginning
Problem Understanding	20%	Clearly interprets problem, identifies all parameters and requirements accurately	Understands problem with minor gaps	Partial understanding; misses key elements	Misinterprets or unable to understand problem
Method Selection	20%	Selects most appropriate method/formula with strong justification	Selects correct method with minor errors	Inappropriate or partially correct method	Unable to select method
Solution Process	25%	Logical, systematic steps with correct approach throughout	Mostly correct steps with minor mistakes	Disorganized steps; multiple errors	No clear process
Accuracy of Calculation	20%	All calculations accurate; correct final answer	Minor calculation errors	Frequent errors; incorrect final answer	No valid calculations
Interpretation of Results	15%	Clearly interprets results with units and engineering relevance	Basic interpretation with minor omissions	Limited or unclear interpretation	No interpretation

1 . The processor uses 8-bit 2's complement representation to perform signed arithmetic operations. The given numbers are +45 and −23. The task is to convert them into binary, perform addition and subtraction, check for overflow, and interpret the final result.

Method Selection

The 2's complement method is used because it is the standard representation for signed integers in computer systems. It allows both addition and subtraction to be performed using the same binary adder, making arithmetic operations simple and efficient.

Solution Process

Binary Conversion

$$+45 = 00101101$$

$$+23 = 00010111$$

2's complement of 23:

$$1's \text{ complement} = 11101000$$

$$\text{Add } 1 = 11101001$$

Therefore,

$$-23 = 11101001$$

Addition: $+45 + (-23)$

$$\begin{array}{r} 00101101 \\ + 11101001 \\ \hline 1\ 00010110 \end{array}$$

Ignoring the carry,

$$\text{Result} = 00010110 = 22$$

Therefore,

$$45 + (-23) = 22$$

Subtraction: $+45 - (-23)$

Since subtracting a negative number is equivalent to addition:

```

  00101101
+ 00010111
-----
  01000100

```

Result = 01000100 = 68

Therefore,

$$45 - (-23) = 68$$

Accuracy of Calculation

Operation	Binary Result	Decimal Result
45 + (-23)	00010110	22
45 - (-23)	01000100	68

Both calculations are correct and within the valid 8-bit signed integer range.

Interpretation of Results

No overflow occurs because the results (22 and 68) are within the range of -128 to +127. The processor checks the carry into and carry out of the most significant bit to detect overflow. Since they are equal, the overflow flag remains 0.

Conclusion

The processor correctly performs signed arithmetic using 2's complement representation. The final results are:

$$45 + (-23) = 22$$

$$45 - (-23) = 68$$

Both operations are accurate, and no overflow occurs. This demonstrates the efficiency of 2's complement arithmetic in modern computer systems.

2. A 4-bit Carry Look-Ahead Adder (CLA) is used to improve the speed of binary addition. Unlike a Ripple Carry Adder (RCA), the CLA calculates carry bits in advance using Generate (G) and Propagate (P) signals, reducing computation delay. The task is to analyze how these signals are formed, compare the CLA with the Ripple Carry Adder, and explain why CLA is faster.

Method Selection

The Carry Look-Ahead Adder (CLA) method is selected because it computes carry signals

simultaneously instead of waiting for each carry to propagate through every bit. This significantly reduces addition time in high-speed processors.

Solution Process

Consider two 4-bit binary numbers:

$$A = 1010$$

$$B = 0110$$

For each bit:

$$\text{Generate (G)} = A \times B$$

$$\text{Propagate (P)} = A \oplus B$$

Bit	A	B	Generate (G)	Propagate (P)
0	0	0	0	0
1	1	1	1	0
2	0	1	0	1
3	1	0	0	1

Using the Generate and Propagate signals, the CLA computes all carry bits at the same time instead of one after another. The final binary addition is:

$$\begin{array}{r} 1010 \\ + 0110 \\ \hline 10000 \end{array}$$

$$\text{Result} = 10000 \text{ (Decimal 16)}$$

Accuracy of Calculation

The binary addition is correct:

A	B	Result
1010	0110	10000

The Generate and Propagate signals correctly determine the carry bits, producing the correct final sum.

Interpretation of Results

The Ripple Carry Adder generates carries sequentially, causing a delay because each bit must wait for the previous carry. In contrast, the Carry Look-Ahead Adder calculates all carry bits simultaneously, making addition much faster. Due to its high speed and reduced delay, the CLA is widely used in modern processors and high-performance computing systems.

Conclusion

The Carry Look-Ahead Adder improves processor performance by reducing carry propagation delay. Compared to the Ripple Carry Adder, it performs binary addition more efficiently, making it suitable for fast arithmetic operations in modern computer architecture.

3. A processor uses Booth's Algorithm to multiply signed binary numbers. The given numbers are -6 and $+5$. The algorithm uses an Accumulator (A), Multiplier (Q), Multiplicand (M), and Booth Bit (Q-1) to perform multiplication efficiently. The task is to explain the multiplication process and show how Booth's algorithm reduces the number of addition and subtraction operations.

Method Selection

Booth's Algorithm is selected because it efficiently handles signed numbers represented in 2's complement form. It minimizes the number of addition and subtraction operations by checking the last bit of the multiplier and the Booth bit before each arithmetic shift.

Solution Process

Given:

Multiplicand (M) = -6

Multiplier (Q) = $+5$

Binary representation (4-bit):

M = 1010 (-6)

Q = 0101 ($+5$)

Accumulator (A) = 0000

Booth Bit (Q-1) = 0

The processor compares the last bit of Q and Q-1:

10 $\rightarrow$ A = A - M

01 $\rightarrow$ A = A + M

00 or 11 $\rightarrow$ No operation

After each operation, an arithmetic right shift is performed on A, Q, and Q-1. This process continues until all bits are processed.

Final binary result:

11100010

Decimal result:

−30

Therefore,

$$-6 \times 5 = -30$$

Accuracy of Calculation

Multiplicand	Multiplier	Binary Result	Decimal Result
−6 +5	11100010		−30

The multiplication is performed correctly using Booth's algorithm.

Interpretation of Results

Booth's Algorithm efficiently multiplies signed numbers by reducing unnecessary addition and subtraction operations. It uses arithmetic right shifts and the Booth bit to simplify the multiplication process. Compared to traditional binary multiplication, Booth's Algorithm is faster and requires fewer arithmetic operations, making it suitable for modern processors.

Conclusion

Booth's Algorithm correctly multiplies −6 and +5, producing the result −30. Its ability to efficiently handle signed numbers and reduce arithmetic operations makes it an important multiplication technique in computer architecture.

4 .A processor performs binary division using Restoring and Non-Restoring Division algorithms. The given example is $13 \div 3$. The task is to explain both methods, compare their working, and identify why the Non-Restoring Division algorithm is preferred in modern processors.

Method Selection

The Restoring Division and Non-Restoring Division algorithms are selected because they are standard hardware methods used to perform binary division. Both generate the quotient and remainder, but the Non-Restoring method reduces the number of operations, making it more efficient.

Solution Process

Given:

$$\text{Dividend} = 13 = 1101_2$$

$$\text{Divisor} = 3 = 0011_2$$

Restoring Division

Subtract the divisor from the remainder.

If the remainder becomes negative, restore the previous value by adding the divisor back.

Repeat the process until all bits are processed.

Result:

Quotient = 0100_2 (4)

Remainder = 0001_2 (1)

Non-Restoring Division

Subtract or add the divisor based on the sign of the remainder.

No restoration step is required.

Continue the process until the final quotient and remainder are obtained.

Result:

Quotient = 0100_2 (4)

Remainder = 0001_2 (1)

Accuracy of Calculation

Method	Quotient	Remainder
Restoring Division	0100_2 (4)	0001_2 (1)
Non-Restoring Division	0100_2 (4)	0001_2 (1)

Both methods produce the correct result:

$$13 \div 3 = 4, \text{ Remainder} = 1$$

Interpretation of Results

Both algorithms generate the same quotient and remainder. However, the Restoring Division method requires an extra restoration step whenever the remainder becomes negative, increasing execution time.

The Non-Restoring Division algorithm eliminates this step, making it faster and more efficient.

Therefore, it is commonly used in modern processors.

Conclusion

Both Restoring and Non-Restoring Division algorithms correctly divide 13 by 3, producing a quotient of 4 and a remainder of 1. Among the two methods, Non-Restoring Division is preferred because it reduces the number of operations and improves processor performance.

5 .The IEEE 754 standard is used to represent real numbers in computers. The given decimal number is -13.25 . The task is to represent it in single precision (32-bit) and double precision (64-bit) format, and explain how floating-point addition is performed, including normalization, rounding, and precision loss.

Method Selection

The IEEE 754 floating-point standard is selected because it provides a standard method for storing and performing arithmetic operations on real numbers. It divides the number into Sign, Exponent, and Mantissa (Fraction) fields.

Solution Process

Given:

Decimal Number = -13.25

Binary form:

$$13.25 = 1101.01_2$$

Normalized form:

$$1.10101 \times 2^3$$

Single Precision (32-bit)

Sign bit = 1 (Negative number)

Exponent = $3 + 127 = 130 = 10000010_2$

Mantissa = 101010000000000000000000

Double Precision (64-bit)

Sign bit = 1

Exponent = $3 + 1023 = 1026 = 10000000010_2$

Mantissa = 10101000000000000000000000000000000000000000

During floating-point addition:

The exponents of both numbers are aligned.

The mantissas are added.

The result is normalized.

Rounding is applied if necessary.

Small precision loss may occur due to limited mantissa bits.

Accuracy of Calculation

